

Brent Foxworth Programming Assignment: Task List (ARRAY)

Program Design

The purpose of this program is to allow a user to manage a simple task list using a fixed-size array that can store up to 8 tasks. The program is menu-driven and allows the user to repeatedly select actions until they choose to quit.

The user can perform the following six actions:

1. Add a task at a specified position
2. Delete a task from a specified position
3. Print the number of tasks currently in the list
4. Print all tasks in numerical order
5. Print all tasks in reverse numerical order
6. Exit the program

Rules and Constraints

- Tasks may only be added to a position if:
 - The position is between 1 and SIZE
 - All previous positions already contain tasks (except position 1)
- If a task is added to an occupied position, all tasks at and after that position are shifted right by one
- Tasks may only be deleted from positions that currently contain a task
- When a task is deleted, all tasks after that position are shifted left
- The program uses procedural (functional) programming, not object-oriented programming

Functions and Their Purpose

Menu and Control Functions

- void displayMenu(): Displays all menu options to the user.
- string getOption(): Prompts the user for a menu option and returns the input as a string.
- int optionValid(string list[SIZE]): Validates the menu option input and ensures it is within range.
- int performOption(string list[SIZE]): Executes the user's selected option and calls the appropriate functions.

Task Management Functions

- `void addTask(string list[SIZE], string task, int p)`: Adds a task at the specified position, shifting tasks if necessary.
- `void deleteTask(string list[SIZE], int q)`: Deletes a task at the specified position and shifts remaining tasks left.
- `void shift(string list[SIZE], int index)`: Shifts tasks to the right starting from a given index.

Input Validation Functions

- `void addValidation(string list[SIZE])`: Validates input for adding a task and calls `addTask()`.
- `void deleteValidation(string list[SIZE])`: Validates input for deleting a task and calls `deleteTask()`.

Utility and Output Functions

- `void printNumTask(string list[SIZE])`: Prints the number of tasks in the list.
- `void printAllTask(string list[SIZE])`: Prints all tasks in numerical order.
- `void reversePrint(string list[SIZE])`: Prints all tasks in reverse numerical order.
- `int stringToInt(string input)`: Converts a numeric string into an integer.
- `int stringSize(string input)`: Returns the length of a string without using `.size()`.
- `bool isEmpty(string list[SIZE], int i)`: Checks whether a given index in the list is empty.

Variables Used and Their Purpose

- **const int SIZE**: Stores the maximum number of tasks allowed.
- **string list[SIZE]**: Primary data structure holding all tasks.
- **int option**: Stores the user's selected menu option.
- **string input**: Temporarily stores user input for validation.
- **string task**: Stores the task name entered by the user.
- **int position**: Stores the position for adding or deleting tasks.
- **bool validInput**: Indicates whether user input has passed validation.
- **bool isInteger**: Indicates whether a string contains only numeric characters.

Program Flow Description

`main()` initializes an empty task list and repeatedly:

- Displays the menu using `displayMenu()`
- Calls `performOption()` to process the user's selection

`performOption()`:

- Calls `optionValid()` to obtain and validate the user's menu selection

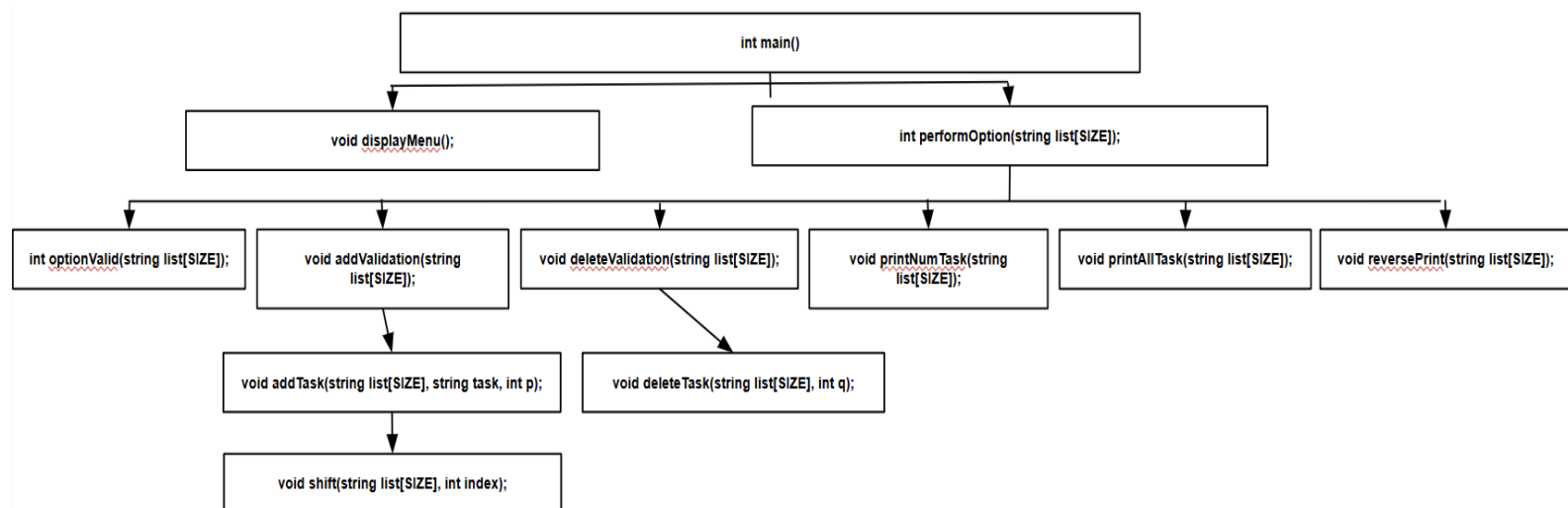
- Uses a switch statement to execute the selected option

Depending on the option selected:

- Add Task (Option 1)
performOption() calls addValidation(), which:
 - Validates the task name and position
 - Converts string input to integers when needed
 - Calls addTask() to insert the task
 - Prints the updated task count and task list (forward and reverse)
- Delete Task (Option 2)
performOption() calls deleteValidation(), which:
 - Ensures the list is not empty
 - Validates the delete position
 - Calls deleteTask() to remove the task
 - Prints the updated task count and task list (forward and reverse)
- Print Number of Tasks (Option 3)
performOption() calls printNumTask() directly
- Print Tasks in Order (Option 4)
performOption() checks if the list is empty and then calls printAllTask()
- Print Tasks in Reverse Order (Option 5)
performOption() checks if the list is empty and then calls reversePrint()
- Quit (Option 6)
The program displays an exit message and terminates the loop

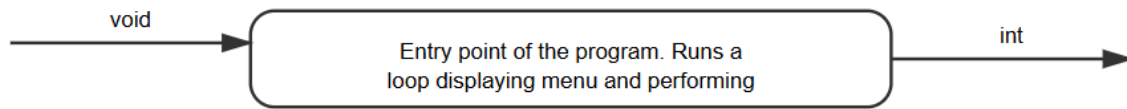
The program continues executing this cycle until the user selects Option 6 (Quit).

Hierarchical diagrams/ Structure charts

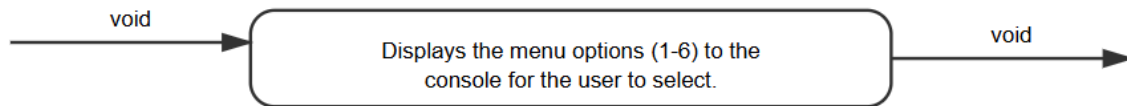


Procedural Specification

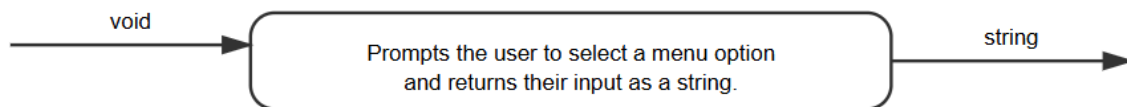
Function main



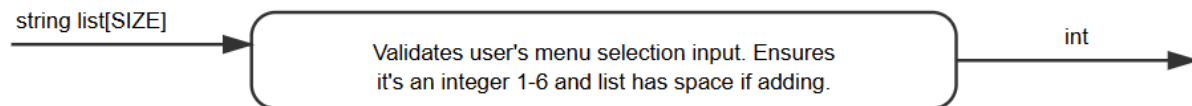
Function displayMenu



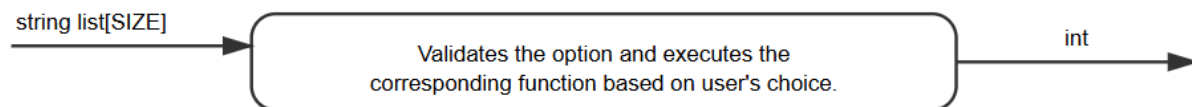
Function getOption



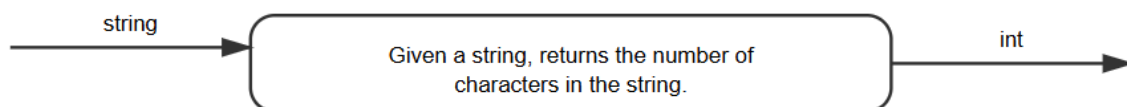
Function optionValid



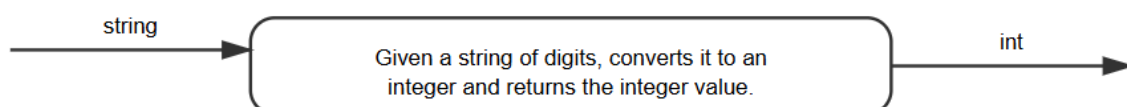
Function performOption

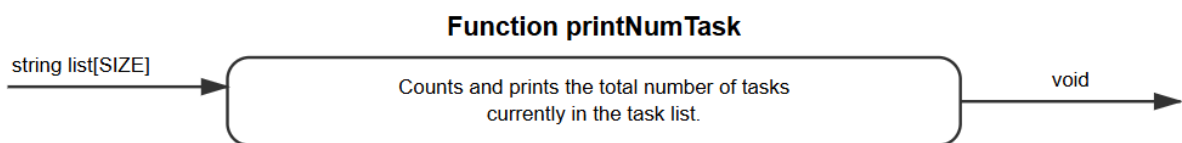
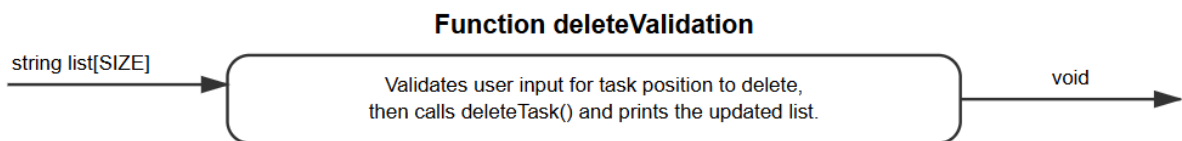
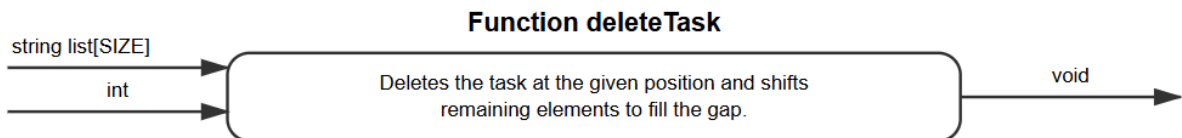
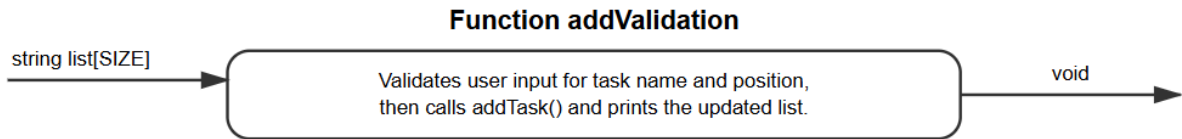
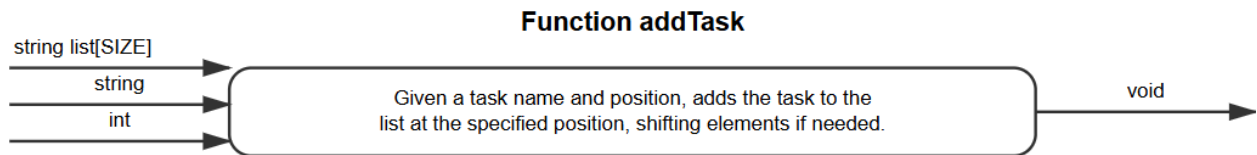
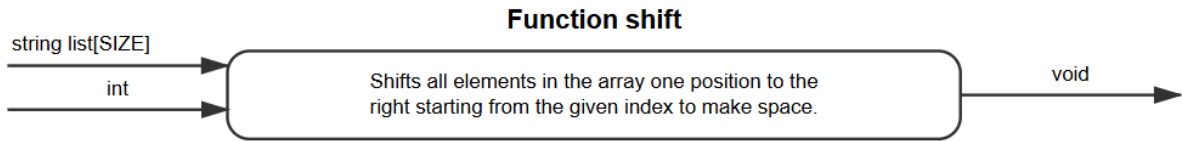
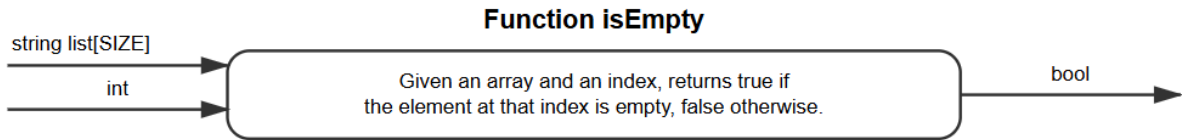


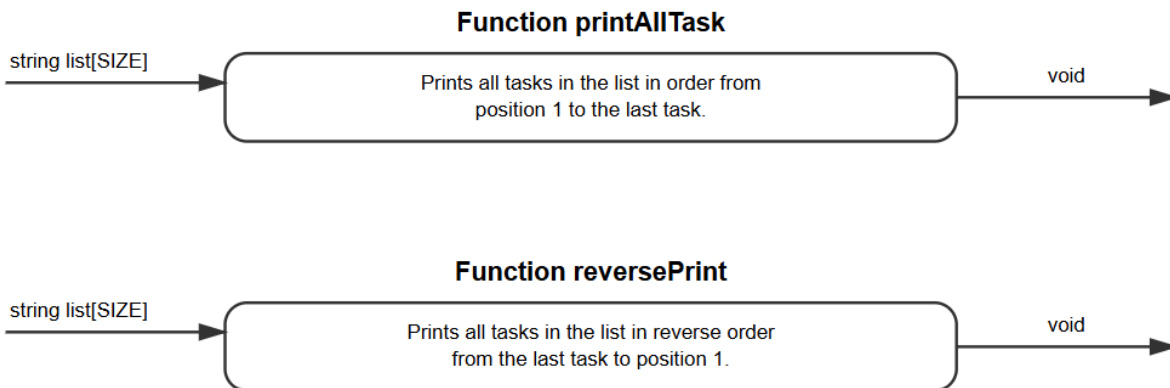
Function stringSize



Function stringToint







Program Listing

```
#include <iostream>
#include <string>
#include <sstream>

using namespace std;

const int SIZE = 8;

void displayMenu();
string getOption();
int optionValid(string list[SIZE]);
void shift(string list[SIZE], int index);
void addTask(string list[SIZE], string task, int p);
void addValidation(string list[SIZE]);
void deleteTask(string list[SIZE], int q);
void deleteValidation(string list[SIZE]);
void printNumTask(string list[SIZE]);
void printAllTask(string list[SIZE]);
void reversePrint(string list[SIZE]);
int performOption(string list[SIZE]);
int stringToInt(string input);
int stringSize(string input);
bool isEmpty(string list[SIZE], int i);
```

```

int main()
{
    string list[SIZE];
    int option = 7;
    while(option != 6)
    {
        displayMenu();
        option = performOption(list);
    }

    return 0;
}

//Converts string variables into integer variables
int stringToint(string input)
{
    int i = 0;
    for (int c = 0; c < stringSize(input); c++)
    {
        i = i * 10 + (input[c] - '0');
    }
    return i;
}

//Displays the menu options
void displayMenu()
{
    cout << "1. Add task X to the list at a particular position p" << endl;
    cout << "2. Delete task X from the task list at position q" << endl;
    cout << "3. Print the number of tasks in the task list" << endl;
    cout << "4. Print all tasks in the task list in the order of position" << endl;
    cout << "5. Print all tasks in the task list in reverse order of position" << endl;
    cout << "6. Quit" << endl;
}

```

```

}

//Get the uses option for the menu and returns it
string getOption ()
{
    string input;
    cout << "Please select an option and enter the corosponding number:
";
    getline(cin >> ws, input);
    return input;
}

//Check if the user input is valid for the menu options then executes the
menu option
int optionValid(string list[SIZE])
{
    bool validInput = false;
    bool isInteger;
    int option;
    while (!validInput)
    {
        string input;
        input = getOption();

        isInteger = true;
        for (int i =0; i < stringSize(input); i++)
        {
            if(!isdigit(input[i]))
            {
                isInteger = false;
                break;
            }
        }

        if (!isInteger)
        {
            cout << "Invalid input. Please enter an integer 1-6" << endl;
        }
    }
}

```



```

        else
        {
            option = stringToint(input);
            if (option == 1 && !isEmpty(list, SIZE-1))
            {
                cout << "The list is full. You must delete an item before
adding another one" << endl;
            }
            else if (option < 1 || option > 6)
            {
                cout << "Invalid input. You must select an option between
1-6" << endl;
            }
            else
            {
                validInput = true;
            }
        }
    }
    return option;
}

//Shifts everything in list over by one at the index to make space for new
task
void shift(string list[SIZE], int index)
{
    for(int i = SIZE-2; i >= index; i--)
    {
        list[i+1] = list[i];
    }
}

//adds new task to the list
void addTask(string list[SIZE], string task, int p)
{
    int index = p - 1;
    if (!isEmpty(list, index))
    {

```

```

        shift(list, index);
    }
    list[index] = task;
}

//validates the user input for the task position and calls addTask
void addValidation(string list[SIZE])
{
    bool validInput = false;
    bool isInteger;
    string line;
    string task;
    string input;
    int position;

    while(!validInput)
    {
        cout << "Enter task name and its position (seprated by a space):
";

        getline(cin >> ws, line);
        stringstream ss(line);
        if (!(ss >> task >> input) || (ss >> ws && !ss.eof()))
        {
            cout << "Invalid input. Please enter exactly: <task>
<position>" <<endl;
            continue;
        }
        isInteger = true;
        for (int i =0; i < stringSize(input); i++)
        {
            if(!isdigit(input[i]))
            {
                isInteger = false;
                break;
            }
        }
        if (!isInteger)
        {
            cout << "Invalid input. Please enter an integer" << endl;

```

```

    }
    else
    {
        position = stringToInt(input);
        if (position < 1 || position > SIZE)
        {
            cout << "Invalid input. Position must be between 1-" <<
SIZE << endl;
        }
        else if (position != 1 && isEmpty(list, position-2))
        {
            cout << "Invalid input. Cannot add a task to a position if
there is not preciding task. Please enter a valid position" << endl;
        }
        else{
            validInput = true;
        }
    }
}

addTask(list, task, position);
printNumTask(list);
printAllTask(list);
reversePrint(list);
}

//Deletes task at index q
void deleteTask(string list[SIZE], int q)
{
    int index = q-1;
    cout << list[index] << " has been deleted" << endl;
    for (int i = index; i < SIZE-1; i++)
    {
        list[i] = list[i+1];
    }
    list[SIZE - 1] = "";
}

```

```

//Print the total number of tasks
void printNumTask(string list[SIZE])
{
    int numTask = 0;
    for (int i=0; i < SIZE; i++)
    {
        if (!isEmpty(list, i))
        {
            numTask++;
        }
    }
    cout << "Number of tasks = " << numTask << endl;
}

```

```

//Prints each task in the list
void printAllTask(string list[SIZE])
{
    for (int i = 0; i < SIZE; i++)
    {
        if (!isEmpty(list, i))
        {
            cout << i+1 << ":" << list[i] << " ";
        }
    }
    cout << endl;
}

```

```

//Prints each task in the list in reverse order
void reversePrint(string list[SIZE])
{
    for (int i = SIZE-1; i >= 0; i--)
    {
        if (!isEmpty(list, i))
        {
            cout << i+1 << ":" << list[i] << " ";
        }
    }
    cout << endl;
}

```

```

}

//Input validation of the deleteTask function. Also calls this function
void deleteValidation(string list[SIZE])
{
    bool validInput = false;
    bool isInteger;
    string line;
    string input;
    int position;

    if (!isEmpty(list, 0))
    {
        while(!validInput)
        {
            cout << "Please enter the position of the task you would like
to delete: ";
            getline(cin >> ws, line);
            stringstream ss(line);
            if (!(ss >> input) || (ss >> ws && !ss.eof()))
            {
                cout << "Invalid input. Please enter exactly one integer."
<< endl;
                continue;
            }
            isInteger = true;
            for (int i =0; i < stringSize(input); i++)
            {
                if(!isdigit(input[i]))
                {
                    isInteger = false;
                    break;
                }
            }
            if (!isInteger)
            {
                cout << "Invalid input. Please enter an integer" << endl;
            }
            else

```

```

        {
            position = stringToInt(input);
            if (position < 1 || position > SIZE)
            {
                cout << "Invalid input. Position must be between 1-"
<< SIZE << endl;
            }
            else if (isEmpty(list, position-1))
            {
                cout << "The position you entered does not have a
task. Please enter a valid position" << endl;
            }
            else
            {
                deleteTask(list, position);
                printNumTask(list);
                printAllTask(list);
                reversePrint(list);
                validInput =true;
            }
        }
    }

}

else
{
    cout << "The list is empty. You can not delete items from an empty
list" << endl;
}
}

//Call optionValid() to make sure option input is valid them excutes the
users option
int performOption(string list[SIZE])
{
    int option;
    option = optionValid(list);

    switch(option)
    {

```

```
case 1:
    addValidation(list);
    cout << endl;
    break;

case 2:
    deleteValidation(list);
    cout << endl;
    break;

case 3:
    printNumTask(list);
    cout << endl;
    break;

case 4:
    if (isEmpty(list, 0))
    {
        cout << "List is empty" << endl;
    }
    else
    {
        printAllTask(list);
    }
    cout << endl;
    break;

case 5:
    if (isEmpty(list, 0))
    {
        cout << "List is empty" << endl;
    }
    else
    {
        reversePrint(list);
    }
    cout << endl;
    break;

case 6:
```

```

        {
            cout << "Exiting program..." << endl;
            break;
        }

    }

    return option;
}

//Gets the size of a string
int stringSize(string input)
{
    int i = 0;
    while (input[i] != '\0')
    {
        i++;
    }
    return i;
}

//Checks if a index in an array is empty
bool isEmpty(string list[SIZE], int i)
{
    bool emptyPlace;
    if (list[i] == "")
    {
        emptyPlace = true;
    }
    else
    {
        emptyPlace = false;
    }

    return emptyPlace;
}

```


Test Cases:

Main Menu Test Cases:

- 1 - Test if add option runs
- 2 - Test if delete option runs
- 3 - Test if program can accurately print the number of tasks in list
- 4 - Test if program can accurately print the tasks in the list
- 5 - Test if program can accurately print the tasks in the list in reverse
- 6 - Test if program ends

Note: 3-5 must be tested multiple times when list empty and where there a different task in the list

Add task Test Cases

- ab 8 - See if program allow this (should not)
- ab 1 - See if task can be added based on position
- cd 2 - See if task can be added based on position
- ef 3 - See if task can be added based on position
- gh 1 - See if task can be added based on position and push everything else back
- ab 1.5 - See if program crashes because 1.5 isn't a valid position
- ab ab - See if program crashes because ab isn't a valid position
- ab 0 - See if program crashes because 0 is not a valid position
- 9 ab - See if program crashes because the max task should be 8
- ab _ - See if program crashes because _ is not a valid position
- ab ?? - See if program crashes because ?? is not a valid position
- 1 2 3 4 - See if program crashes because this input isn't valid
- 1 ab - See if program crashes because this input isn't valid

Delete task Test Cases

Notes: for this example the current list is 1: ab 2:cd 3:ef 4:gh

- 1 - See if ab is deleted and all other task are moved up 1
- 5 - See if program can delete at empty position (should not)
- 0 - See if program can delete from a position that does not exist (should not)
- 9 - See if program can delete from a position that does not exist (should not)
- A - See if program crashes because "A" is not a valid position
- _ - See if program crashes because "_" is not a valid position
- 1 2 3 - See if program crashes because this input isn't valid
- 1 ab - See if program crashes because this input isn't valid
- ab 1 - See if program crashes because this input isn't valid

Results of Test Cases

Main Menu Test Cases:

```
1. Add task X to the list at a particular position p
2. Delete task X from the task list at position q
3. Print the number of tasks in the task list
4. Print all tasks in the task list in the order of position
5. Print all tasks in the task list in reverse order of position
6. Quit
Please select an option and enter the corresponding number: 1
Enter task name and its position (separated by a space): |
```

```
1. Add task X to the list at a particular position p
2. Delete task X from the task list at position q
3. Print the number of tasks in the task list
4. Print all tasks in the task list in the order of position
5. Print all tasks in the task list in reverse order of position
6. Quit
Please select an option and enter the corresponding number: 2
Please enter the position of the task you would like to delete: |
```

```
1. Add task X to the list at a particular position p
2. Delete task X from the task list at position q
3. Print the number of tasks in the task list
4. Print all tasks in the task list in the order of position
5. Print all tasks in the task list in reverse order of position
6. Quit
Please select an option and enter the corresponding number: 3
Number of tasks = 3
```

```
1. Add task X to the list at a particular position p
2. Delete task X from the task list at position q
3. Print the number of tasks in the task list
4. Print all tasks in the task list in the order of position
5. Print all tasks in the task list in reverse order of position
6. Quit
Please select an option and enter the corresponding number: 4
1:ab 2:cd 3:ef
```

```
1. Add task X to the list at a particular position p
2. Delete task X from the task list at position q
3. Print the number of tasks in the task list
4. Print all tasks in the task list in the order of position
5. Print all tasks in the task list in reverse order of position
6. Quit
Please select an option and enter the corresponding number: 5
3:ef 2:cd 1:ab
```

Add task Test Cases:

```
1. Add task X to the list at a particular position p
2. Delete task X from the task list at position q
3. Print the number of tasks in the task list
4. Print all tasks in the task list in the order of position
5. Print all tasks in the task list in reverse order of position
6. Quit
Please select an option and enter the corresponding number: 1
Enter task name and its position (separated by a space): ab 8
Invalid input. Cannot add a task to a position if there is not preceding task. Please enter a valid position
Enter task name and its position (separated by a space): ab 1
Number of tasks = 1
1:ab
1:ab
```

```
1. Add task X to the list at a particular position p
2. Delete task X from the task list at position q
3. Print the number of tasks in the task list
4. Print all tasks in the task list in the order of position
5. Print all tasks in the task list in reverse order of position
6. Quit
Please select an option and enter the corresponding number: 1
Enter task name and its position (separated by a space): cd 2
Number of tasks = 2
1:ab 2:cd
2:cd 1:ab
```

```
1. Add task X to the list at a particular position p
2. Delete task X from the task list at position q
3. Print the number of tasks in the task list
4. Print all tasks in the task list in the order of position
5. Print all tasks in the task list in reverse order of position
6. Quit
Please select an option and enter the corresponding number: 1
Enter task name and its position (separated by a space): ef 3
Number of tasks = 3
1:ab 2:cd 3:ef
3:ef 2:cd 1:ab
```

```
1. Add task X to the list at a particular position p
2. Delete task X from the task list at position q
3. Print the number of tasks in the task list
4. Print all tasks in the task list in the order of position
5. Print all tasks in the task list in reverse order of position
6. Quit
Please select an option and enter the corresponding number: 1
Enter task name and its position (separated by a space): gh 1
Number of tasks = 4
1:gh 2:ab 3:cd 4:ef
4:ef 3:cd 2:ab 1:gh
```

```
1. Add task X to the list at a particular position p
2. Delete task X from the task list at position q
3. Print the number of tasks in the task list
4. Print all tasks in the task list in the order of position
5. Print all tasks in the task list in reverse order of position
6. Quit
Please select an option and enter the corresponding number: 1
Enter task name and its position (separated by a space): ab 1.5
Invalid input. Please enter an integer
Enter task name and its position (separated by a space): ab ab
Invalid input. Please enter an integer
Enter task name and its position (separated by a space): ab 0
Invalid input. Position must be between 1-8
Enter task name and its position (separated by a space): 9 ab
Invalid input. Please enter an integer
Enter task name and its position (separated by a space): ab __
Invalid input. Please enter an integer
Enter task name and its position (separated by a space): ab 9
Invalid input. Position must be between 1-8
Enter task name and its position (separated by a space): ab ??
Invalid input. Please enter an integer
Enter task name and its position (separated by a space): 1 2 3 4
Invalid input. Please enter exactly: <task> <position>
Enter task name and its position (separated by a space): 1 ab
Invalid input. Please enter an integer
Enter task name and its position (separated by a space): |
```

Delete task Test Cases:

```
1. Add task X to the list at a particular position p
2. Delete task X from the task list at position q
3. Print the number of tasks in the task list
4. Print all tasks in the task list in the order of position
5. Print all tasks in the task list in reverse order of position
6. Quit
Please select an option and enter the corresponding number: 2
Please enter the position of the task you would like to delete: 1
ab has been deleted
Number of tasks = 3
1:cd 2:ef 3:gh
3:gh 2:ef 1:cd
```

```
1. Add task X to the list at a particular position p
2. Delete task X from the task list at position q
3. Print the number of tasks in the task list
4. Print all tasks in the task list in the order of position
5. Print all tasks in the task list in reverse order of position
6. Quit
Please select an option and enter the corresponding number: 2
Please enter the position of the task you would like to delete: 5
The position you entered does not have a task. Please enter a valid position
Please enter the position of the task you would like to delete: 0
Invalid input. Position must be between 1-8
Please enter the position of the task you would like to delete: 9
Invalid input. Position must be between 1-8
Please enter the position of the task you would like to delete: A
Invalid input. Please enter an integer
Please enter the position of the task you would like to delete: _
Invalid input. Please enter an integer
Please enter the position of the task you would like to delete: 1 2 3
Invalid input. Please enter exactly one integer.
Please enter the position of the task you would like to delete: 1 ab
Invalid input. Please enter exactly one integer.
Please enter the position of the task you would like to delete: ab 1
Invalid input. Please enter exactly one integer.
Please enter the position of the task you would like to delete: |
```